ARTIFICIAL INTELLIGENCE INTERNSHIP PROJECT REPORT

BREAST CANCER CLASSIFICATION USING NEURAL NETWORKS

A project report submitted in fulfilment of the requirements of the Artificial Intelligence Internship Programme (September 2026 batch)

Submitted by

SANGINI AGRAWAL

NAVIOTECH SOLUTION

Artificial Intelligence Domain

October 2026

CERTIFICATE

This is to certify that the project report entitled **"Breast Cancer Classification using Neural Networks"** submitted by **Sangini Agrawal** is a record of bona fide work carried out under the Artificial Intelligence Internship Programme (September 2026 batch) of Naviotech Solution.

The work presented in this report has been carried out in accordance with the project guidelines issued by the organisation.

DECLARATION

I, **Sangini Agrawal**, hereby declare that the project report entitled **"Breast Cancer Classification using Neural Networks"** submitted to Naviotech Solution as part of the Artificial Intelligence Internship Programme is a record of my own work. The dataset, software libraries and published works used in this project have been duly acknowledged and cited in the report.

Sangini Agrawal
AI Intern, September 2026 batch

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to **Naviotech Solution** for giving me the opportunity to carry out this project as part of the Artificial Intelligence Internship Programme. I am thankful to the Academic Head and the academic team for the project allocation session, the project guide manual and the starter material, which provided a clear structure for the work.

I thank my project guide and mentors for their guidance on project development, documentation and submission, and the support team for promptly answering questions during the internship.

I also acknowledge Dr William H. Wolberg, W. Nick Street and Olvi L. Mangasarian of the University of Wisconsin for creating the Breast Cancer Wisconsin (Diagnostic) dataset and making it publicly available through the UCI Machine Learning Repository, and the developers of TensorFlow, Keras and scikit-learn, whose open-source tools made this work possible.

Finally, I am grateful to my family and friends for their constant encouragement and support.

Sangini Agrawal

ABSTRACT

Breast cancer is the most common cancer in women worldwide, and finding it early greatly improves survival. This project develops an artificial neural network that classifies breast tumours as benign or malignant from 30 numeric features describing the cell nuclei in a fine-needle aspirate image. The Breast Cancer Wisconsin (Diagnostic) dataset of 569 patients was explored, checked for missing values and duplicates, divided into stratified training (364), validation (91) and test (114) sets, and standardised using statistics from the training set only to avoid data leakage.

A feed-forward network with two hidden layers of 16 and 8 ReLU neurons, 20% dropout and a two-neuron softmax output (650 trainable parameters) was implemented in TensorFlow/Keras. It was trained with the Adam optimiser and sparse categorical cross-entropy loss, using early stopping on the validation loss. A few network sizes were tried and gave similar results, so the smallest stable two-layer network was chosen.

On the unseen test set the model achieved an accuracy of 96.49%, a sensitivity (recall for malignant tumours) of 97.62%, a precision of 93.18%, an F1-score of 95.35% and a ROC-AUC of 0.994, detecting 41 of 42 cancers. Five-fold cross-validation over all patients gave a mean accuracy of 97.72% ± 0.89%, the most consistent of five models compared, which also included logistic regression, a support vector machine, k-nearest neighbours and a random forest. 103 of the 104 highly confident predictions were correct; three of the four errors were borderline cases, while one cancer was missed with high confidence, so the model is best used as a second reader alongside a specialist. The project meets its target of more than 90% accuracy and provides a reproducible notebook, modular Python code and a command-line prediction tool.

Keywords: *breast cancer, neural networks, deep learning, TensorFlow, Keras, classification, Wisconsin Diagnostic dataset, medical diagnosis*

TABLE OF CONTENTS

CERTIFICATE i
DECLARATION ii
ACKNOWLEDGEMENT iii
ABSTRACT iv
LIST OF FIGURES vii
LIST OF TABLES viii
LIST OF ABBREVIATIONS ix
CHAPTER 1 INTRODUCTION 1
1.1 Background 1
1.2 Artificial Intelligence in Medical Diagnosis 1
1.3 Problem Statement 2
1.4 Objectives 2
1.5 Scope of the Project 2
1.6 Applications 2
1.7 Organisation of the Report 3
CHAPTER 2 LITERATURE REVIEW 4
2.1 Origins of the Wisconsin Diagnostic Dataset 4
2.2 Classical Machine Learning on the Dataset 4
2.3 Neural Networks and Deep Learning 4
2.4 Deep Learning in Cancer Diagnosis 5
2.5 Summary of Related Work 5
2.6 Research Gap and Contribution 6
CHAPTER 3 METHODOLOGY 7
3.1 Overall Workflow 7
3.2 Dataset Description 7
3.3 Data Cleaning 8
3.4 Exploratory Data Analysis 8
3.5 Data Preprocessing 11
3.6 Neural Network Fundamentals 12
3.7 Evaluation Metrics 13
3.8 Cross-Validation and Baseline Models 13
CHAPTER 4 SYSTEM DESIGN 15
4.1 System Architecture 15
4.2 Neural Network Architecture 15
4.3 Hyper-parameters 16
4.4 Choice of Architecture 16
4.5 Software and Hardware Requirements 16
4.6 Software Design and Project Structure 17
CHAPTER 5 IMPLEMENTATION 18

5.1 Environment Setup....18
5.2 Data Collection and Cleaning....18
5.3 Data Preprocessing....18
5.4 Building the Neural Network....19
5.5 Training the Model....19
5.6 Evaluation....20
5.7 Predictive System....21
5.8 Saving the Model....21
CHAPTER 6 RESULTS AND DISCUSSION....23
6.1 Training Behaviour....23
6.2 Performance on the Test Set....23
6.3 Confusion Matrix....24
6.4 ROC Curve....24
6.5 Prediction Confidence and Error Analysis....25
6.6 Comparison with Classical Machine Learning....26
6.7 Cross-Validation Results....27
6.8 Comparison with Published Work....28
6.9 Discussion....28
6.10 Limitations....28
CHAPTER 7 CONCLUSION....30
CHAPTER 8 FUTURE SCOPE....31
REFERENCES....32
APPENDIX A SOURCE CODE....34

LIST OF FIGURES

Figure 3.1: Class distribution of the 569 samples.....9
Figure 3.2: Distributions of six key features for benign and malignant tumours.....10
Figure 3.3: Correlation between the ten "mean" features and the label.....11
Figure 4.1: System architecture from raw data to diagnosis.....15
Figure 4.2: Neural network architecture.....15
Figure 5.1: Output: first rows of the dataset (selected columns).....18
Figure 5.2: Output: model summary.....19
Figure 5.3: Output: training log (first and last epochs).....20
Figure 5.4: Output: test metrics and classification report.....21
Figure 5.5: Output: predictions for a benign and a malignant sample.....21
Figure 6.1: Training and validation accuracy and loss per epoch.....23
Figure 6.2: Confusion matrix on the test set.....24
Figure 6.3: ROC curve for detecting malignant tumours.....25
Figure 6.4: Distribution of predicted malignancy probabilities on the test set.....26
Figure 6.5: Accuracy, F1-score and sensitivity of all models on the test set.....27

LIST OF TABLES

Table 2.1: Summary of related work.....5
Table 3.1: The seven phases of the project.....7
Table 3.2: The ten nucleus properties.....7
Table 3.3: Dataset summary.....8
Table 3.4: Average value of the ten "mean" features per class.....9
Table 3.5: Data split (stratified, random seed 42).....11
Table 3.6: Evaluation metrics (malignant = positive class).....13
Table 4.1: Layer-by-layer structure and parameter count.....16
Table 4.2: Training hyper-parameters.....16
Table 4.3: Software requirements.....16
Table 4.4: Hardware requirements.....17
Table 4.5: Modules and responsibilities.....17
Table 6.1: Test-set performance of the neural network.....23
Table 6.2: Classification report per class.....24
Table 6.3: Misclassified test patients.....26
Table 6.4: Test-set comparison with baseline models.....26
Table 6.5: 5-fold stratified cross-validation accuracy (all 569 patients).....27
Table 6.6: Comparison with a published study on the same dataset.....28

LIST OF ABBREVIATIONS

AI	Artificial Intelligence
ML	Machine Learning
NN	Neural Network
MLP	Multilayer Perceptron
FNA	Fine-Needle Aspirate
ReLU	Rectified Linear Unit
Adam	Adaptive Moment Estimation (optimiser)
CV	Cross-Validation
ROC	Receiver Operating Characteristic
AUC	Area Under the (ROC) Curve
TP / TN	True Positive / True Negative
FP / FN	False Positive / False Negative
SVM	Support Vector Machine
RBF	Radial Basis Function
kNN	k-Nearest Neighbours
UCI	University of California, Irvine (Machine Learning Repository)
WHO	World Health Organization

CHAPTER 1
INTRODUCTION

1.1 Background

Breast cancer is the most frequently diagnosed cancer among women worldwide. According to the World Health Organization, an estimated 2.4 million women were diagnosed with breast cancer in 2024 and about 694,000 women died from the disease in the same year [1]. Survival depends heavily on access to care: fewer than one in two women (41.9%) in low-income countries survive five years after diagnosis, compared with nearly nine in ten (87.3%) in high-income countries [1]. The WHO also points out that deaths go down when breast cancer is found and treated early, so a fast and reliable diagnosis really matters.

A common first step in diagnosing a suspicious breast lump is a **fine-needle aspirate (FNA)**: a thin needle draws a small sample of cells from the mass, which is stained and examined under a microscope. A cytopathologist then judges whether the cells look benign (non-cancerous) or malignant (cancerous) based on the size, shape and texture of the cell nuclei. This depends a lot on the expert's experience, it takes time, and two experts may not always agree.

In the early 1990s researchers at the University of Wisconsin showed that the appearance of cell nuclei in a digitised FNA image can be summarised by a small set of numeric measurements, and that a computer can learn to separate benign from malignant samples using those measurements [2], [3]. The Breast Cancer Wisconsin (Diagnostic) dataset that came out of this work is now one of the most commonly used datasets for learning medical machine learning.

1.2 Artificial Intelligence in Medical Diagnosis

Artificial intelligence (AI), and machine learning (ML) in particular, allows a computer to learn decision rules from examples instead of being programmed with them explicitly. Given many labelled patient records, a learning algorithm can discover which combinations of measurements are associated with disease and apply that knowledge to new patients.

Deep learning, a branch of ML based on artificial neural networks with several layers, has done very well in medicine. For example, neural networks have matched dermatologists in classifying skin cancer from photographs [4] and have performed comparably to expert radiologists in reading screening mammograms [5]. These systems are not meant to replace clinicians but to act as a consistent *second reader* that highlights cases that need attention.

This project applies the same idea on a much smaller scale: a feed-forward neural network is trained on the Wisconsin Diagnostic dataset to classify tumours as benign or malignant.

1.3 Problem Statement

Given 30 numeric measurements describing the cell nuclei in a fine-needle aspirate of a breast mass, build a system that automatically classifies the tumour as **benign** or **malignant**. The system should have high overall accuracy and, most importantly, a high **sensitivity** for malignant tumours. This is important because telling a cancer patient that their tumour is benign (a false negative) is the worst mistake the system can make.

1.4 Objectives

The objectives of the project, as set out in the Naviotech project guide, are to:

1. Develop an intelligent cancer prediction system based on a neural network.
2. Understand classification algorithms, both neural networks and classical machine-learning methods.
3. Learn healthcare data analysis techniques: exploration, cleaning and preprocessing of real diagnostic data.
4. Apply neural networks to a real-world medical dataset using TensorFlow and Keras.
5. Evaluate model performance with classification metrics: accuracy, precision, recall, F1-score and the confusion matrix.
6. Build a predictive system that accepts a new patient's measurements and returns a diagnosis.

The success criterion given in the guide is a classification accuracy above 90%.

1.5 Scope of the Project

The project covers the full machine-learning workflow on the Wisconsin Diagnostic dataset: data collection, cleaning, exploratory analysis, preprocessing, model design, training, evaluation, comparison with classical algorithms, and a prediction tool usable from a notebook or the command line. It does not cover image processing of raw microscope slides (the dataset already contains extracted measurements) or clinical deployment, which would require regulatory approval and validation on new patient populations.

1.6 Applications

A reliable tumour classifier of this kind can support several real-world uses:

- **Medical diagnosis support systems:** a second opinion that flags suspicious samples for the cytopathologist.

- **Clinical decision-making:** probability scores help prioritise which patients need urgent follow-up or biopsy.
- **Healthcare analytics:** screening programmes can monitor case mix and workload across clinics.
- **Disease prediction systems:** the same pipeline can be retrained for other tabular diagnostic datasets.
- **Research and development:** a simple, reproducible starting point for comparing new algorithms.

1.7 Organisation of the Report

Chapter 2 reviews related work. Chapter 3 describes the dataset and methodology, including the theory behind the neural network and the evaluation metrics. Chapter 4 presents the system design and the choice of network architecture. Chapter 5 explains the implementation with code excerpts and program outputs. Chapter 6 reports and discusses the results. Chapters 7 and 8 give the conclusion and directions for future work, followed by the references and an appendix with source code.

CHAPTER 2
LITERATURE REVIEW

2.1 Origins of the Wisconsin Diagnostic Dataset

Street, Wolberg and Mangasarian [2] developed an interactive image-analysis system in which an operator outlines a few cell nuclei in a digitised FNA image and the software fits *active contour* models ("snakes") to their boundaries. Ten properties of each nucleus are computed from these contours (radius, texture, perimeter, area, smoothness, compactness, concavity, concave points, symmetry and fractal dimension), and for every image the mean, the standard error and the worst (mean of the three largest) values are recorded, giving 30 features per patient.

To classify the samples, the authors used the Multisurface Method-Tree (MSM-T), a linear-programming technique that builds a small decision tree of separating planes [3], based on the robust linear programming formulation of Bennett and Mangasarian [6]. The dataset of 569 patients was donated to the UCI Machine Learning Repository in 1995 [3] and is distributed with the scikit-learn library [7].

2.2 Classical Machine Learning on the Dataset

Because the dataset is small, clean and well documented, it is widely used to compare classification algorithms. Agarap [8] compared six algorithms on a 70/30 train-test split and reported test accuracies of 99.04% for a multilayer perceptron (MLP), 97.66% for softmax regression, 96.09% for both linear regression and a support vector machine (SVM), 94.74% and 93.57% for nearest-neighbour classifiers with L2 and L1 distance, and 93.75% for a GRU-SVM hybrid. The study concluded that all of the algorithms perform well on this dataset, and that the MLP did best on that split.

Classical algorithms used as baselines in this project are well established. Logistic regression models the log-odds of the positive class as a linear function of the features. The support vector machine [9] finds the boundary with the largest margin between classes and, with a radial basis function (RBF) kernel, can model non-linear boundaries. The k-nearest-neighbour rule [10] assigns a new sample the majority label of its k closest training samples. Random forests [11] combine many decision trees trained on random subsets of samples and features.

2.3 Neural Networks and Deep Learning

An artificial neural network is built from layers of simple units (neurons) that compute a weighted sum of their inputs followed by a non-linear activation function. Rumelhart, Hinton and Williams [12] popularised *back-propagation*, the algorithm that computes how

each weight should change to reduce the prediction error. Hornik, Stinchcombe and White [13] proved that a network with even one hidden layer can approximate any continuous function, given enough neurons, which is why neural networks are so flexible.

Several later developments make modern networks practical to train. The rectified linear unit (ReLU) activation [14] avoids the vanishing gradients of older sigmoid units. Dropout [15] randomly disables neurons during training and greatly reduces over-fitting. The Adam optimiser [16] adapts the learning rate of every weight automatically and is now the default choice for most problems. Early stopping, which halts training when performance on a validation set stops improving, is one of the simplest and most effective forms of regularisation [17]. LeCun, Bengio and Hinton [18] and the textbook by Goodfellow, Bengio and Courville [19] give a good overview of the field.

2.4 Deep Learning in Cancer Diagnosis

Deep learning has been applied successfully to many cancer-diagnosis tasks. Esteva et al. [4] trained a convolutional neural network on about 129,000 clinical images and achieved performance on par with board-certified dermatologists in identifying skin cancers. McKinney et al. [5] evaluated an AI system for breast cancer screening on large UK and US mammography datasets and reported reductions in both false positives and false negatives compared with the original radiologist decisions. These studies use raw images and very large datasets. The present project is much smaller: it uses ready-made measurements and checks how well a small network can do on a small dataset.

2.5 Summary of Related Work

Table 2.1: Summary of related work

Study	Data	Method	Key result
Street et al., 1993 [2]	FNA images (Wisconsin)	Active contours + MSM-T	Introduced the 30 nuclear features
Bennett & Mangasarian, 1992 [6]	Linearly inseparable sets	Robust linear programming	Basis of the MSM-T classifier
Agarap, 2018 [8]	Wisconsin Diagnostic, 70/30 split	MLP, SVM, kNN, softmax and others	MLP 99.04% test accuracy
Esteva et al., 2017 [4]	~129,000 skin images	Convolutional neural network	Dermatologist-level accuracy
McKinney et al., 2020 [5]	UK and US mammograms	Deep learning ensemble	Fewer false positives and negatives than original readings
This project	Wisconsin Diagnostic, 64/16/20 split	Neural network 30-16-8-2 + 4 baselines	96.49% test, 97.72% 5-fold CV

2.6 Research Gap and Contribution

Many projects on this dataset report results from just one train-test split. With a small dataset, one or two patients can change the accuracy by more than one percentage point, so a single split can be misleading. Many also report only accuracy. This project tries to avoid these problems. The test set was kept completely separate until the very end, recall and precision for malignant tumours were reported (not just accuracy), and the final result was checked with 5-fold cross-validation against four classical models trained in the same way.

CHAPTER 3
METHODOLOGY

3.1 Overall Workflow

The project follows the seven-phase workflow specified in the Naviotech project guide (Table 3.1). Each phase was implemented in the Jupyter notebook and also as separate Python files, so the whole pipeline can be run again with one command.

Table 3.1: The seven phases of the project

Phase	Activity	Output
1. Data collection	Load the Breast Cancer Wisconsin (Diagnostic) dataset	Table of 569 patients × 31 columns
2. Data cleaning	Check and remove missing values and duplicate rows	Clean dataset
3. Data preprocessing	Stratified train/validation/test split; feature scaling	Standardised feature matrices
4. Neural network development	Design layers and activation functions	Compiled Keras model
5. Model training	Train with Adam and early stopping	Trained weights, training history
6. Evaluation	Accuracy, precision, recall, F1, confusion matrix, ROC, CV, baselines	Metrics, charts, comparison
7. Prediction system	Accept new measurements and predict the diagnosis	Notebook function and command-line tool

3.2 Dataset Description

The Breast Cancer Wisconsin (Diagnostic) dataset [3] contains one row for each of 569 patients. Its features are computed from a digitised image of a fine-needle aspirate of a breast mass and describe characteristics of the cell nuclei present in the image. Ten properties are measured for each nucleus (Table 3.2), and three statistics of each property (mean, standard error and "worst", i.e. the mean of the three largest values) are stored, giving 30 real-valued features.

Table 3.2: The ten nucleus properties

Property	Meaning
Radius	Mean distance from the centre of the nucleus to points on its boundary
Texture	Standard deviation of grey-scale values inside the nucleus
Perimeter	Length of the nucleus boundary
Area	Area enclosed by the boundary
Smoothness	Local variation in radius lengths
Compactness	Perimeter² / area − 1.0
Concavity	Severity of concave (indented) portions of the boundary

Property	Meaning
Concave points	Number of concave portions of the boundary
Symmetry	Symmetry of the nucleus shape
Fractal dimension	"Coastline approximation" − 1: complexity of the boundary

Table 3.3: Dataset summary

Property	Value
Source	UCI Machine Learning Repository (DOI 10.24432/C5DW2B); bundled with scikit-learn
Number of patients (rows)	569
Number of features	30 numeric + 1 label
Classes	Malignant: 212 (37.3%), Benign: 357 (62.7%)
Label encoding (scikit-learn)	0 = Malignant, 1 = Benign
Missing values / duplicate rows	0 / 0

3.3 Data Cleaning

The dataset was checked for missing values and duplicate rows, and none were found. The cleaning step (removing incomplete or repeated rows) was still kept in the code, so it will still work if someone uses a different copy of the data, such as the CSV version on Kaggle. Outliers were not removed, because very large values (for example a very large tumour area) are real and mostly belong to malignant cases.

3.4 Exploratory Data Analysis

Figure 3.1 shows that the classes are moderately imbalanced: 62.7% of the samples are benign and 37.3% malignant. A model that always answered "benign" would already reach 62.7% accuracy, so accuracy alone is not enough, and the recall for malignant tumours must be reported as well. The imbalance also motivates a *stratified* split, which keeps the same class ratio in every subset.

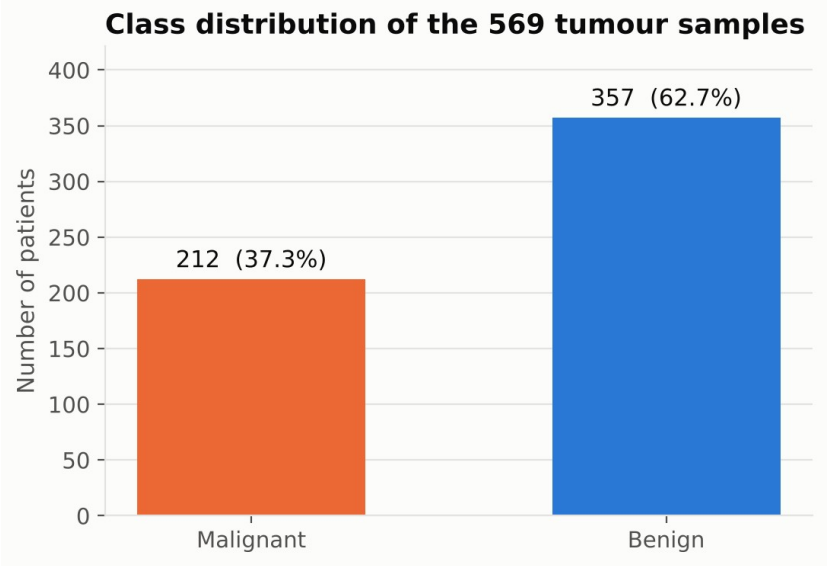


Figure 3.1: Class distribution of the 569 samples

Table 3.4 compares the average value of the ten "mean" features in each class. Malignant tumours have much larger nuclei (mean area 978 versus 463) and far more concave boundaries (mean concave points 0.088 versus 0.026). Fractal dimension, in contrast, is almost identical in both classes.

Table 3.4: Average value of the ten "mean" features per class

Feature	Malignant	Benign
mean radius	17.46	12.15
mean texture	21.60	17.91
mean perimeter	115.37	78.08
mean area	978.38	462.79
mean smoothness	0.103	0.092
mean compactness	0.145	0.080
mean concavity	0.161	0.046
mean concave points	0.088	0.026
mean symmetry	0.193	0.174
mean fractal dimension	0.063	0.063

Figure 3.2 shows the full distribution of six key features. For radius, perimeter, area, concavity and concave points the two classes overlap only slightly, which indicates that a classifier can separate them with high accuracy. Texture shows more overlap and is therefore a weaker individual signal.

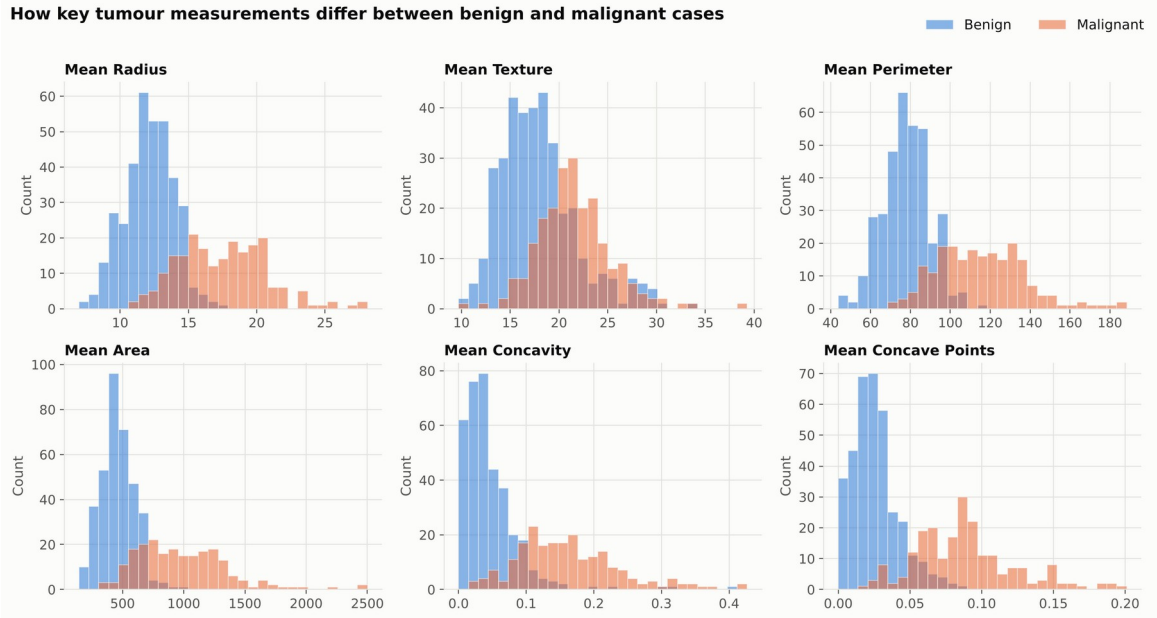


Figure 3.2: Distributions of six key features for benign and malignant tumours

The correlation heatmap in Figure 3.3 shows that many features are strongly related to each other. Radius, perimeter and area are almost perfectly correlated ($r \approx 0.99$) because all three describe the size of the nucleus. Concavity, concave points and compactness also move together. The label is strongly negatively correlated with concave points (−0.78), perimeter (−0.74), radius (−0.73) and area (−0.71). The sign is negative only because label 1 means benign: larger and more irregular nuclei are associated with label 0 (malignant). Neural networks can handle correlated inputs, so all 30 features were kept.

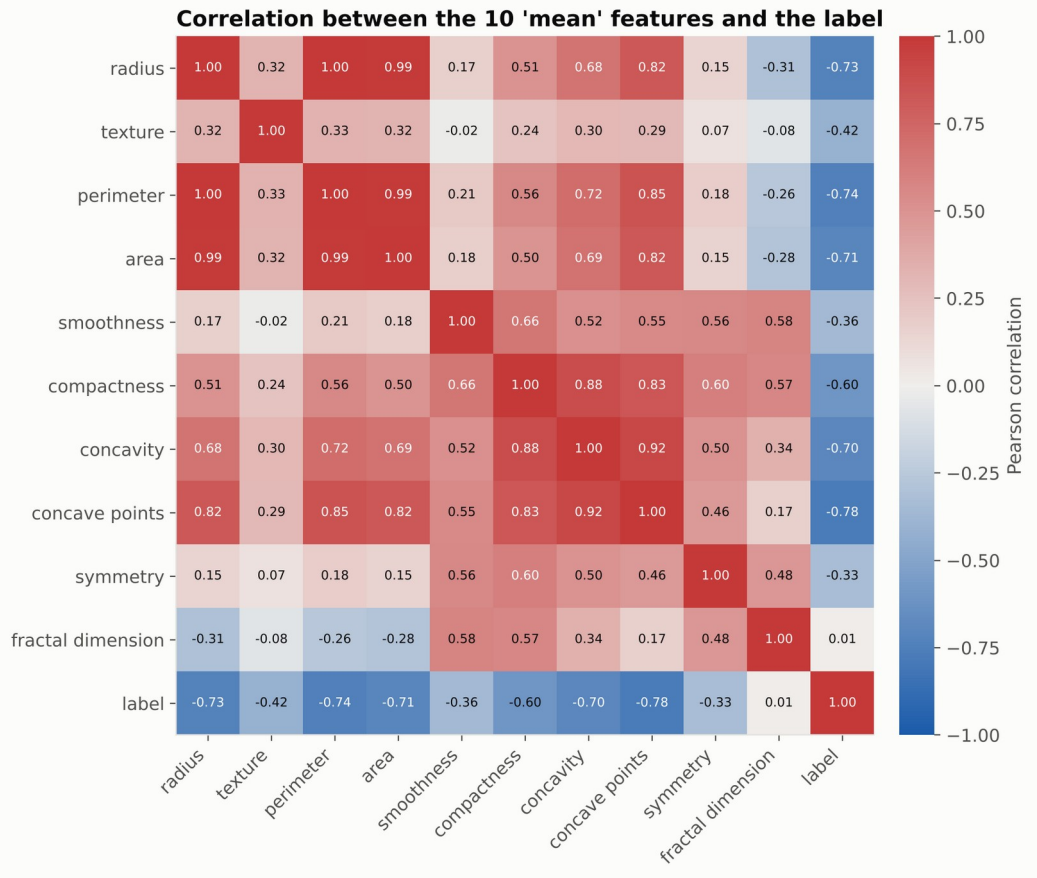


Figure 3.3: Correlation between the ten "mean" features and the label

3.5 Data Preprocessing

3.5.1 Train, validation and test split

The data were divided into three stratified subsets (Table 3.5). The **test set** (20% of patients) was set aside first and used only once, for the final evaluation. From the remaining patients, 20% formed a **validation set** that early stopping monitors during training; the rest formed the **training set** used to fit the weights. A fixed random seed (42) makes the split reproducible.

Table 3.5: Data split (stratified, random seed 42)

Subset	Patients	Share	Malignant	Benign	Purpose
Training	364	64%	136	228	Fit the network weights
Validation	91	16%	34	57	Early stopping
Test	114	20%	42	72	Final, unbiased evaluation
Total	569	100%	212	357	

3.5.2 Feature standardisation

The features have very different scales: mean smoothness ranges from about 0.05 to 0.16, while mean area ranges from about 144 to 2,501. Without rescaling, features with large

values would dominate the weighted sums inside the network and slow down training. Each feature was therefore standardised to zero mean and unit standard deviation:

$$z = (x - \mu) / \sigma$$

where μ and σ are the mean and standard deviation of that feature **in the training set only**. The same μ and σ are then applied to the validation and test sets and to any new patient. If the scaler were fitted on all the data, some information from the test set would leak into training and the results would look better than they really are.

3.5.3 Label encoding

The labels are already integers (0 = malignant, 1 = benign), so they can be used directly with the sparse categorical cross-entropy loss and no one-hot encoding is needed.

3.6 Neural Network Fundamentals

3.6.1 The artificial neuron and dense layers

Each neuron computes a weighted sum of its inputs plus a bias and passes it through an activation function f:

$$a = f(w_1x_1 + w_2x_2 + ... + w_nx_n + b)$$

In a *dense* (fully connected) layer, every neuron receives every output of the previous layer. A layer with n inputs and m neurons therefore has n × m weights and m biases.

3.6.2 Activation functions

The hidden layers use the rectified linear unit, ReLU(z) = max(0, z), which is fast to compute and does not saturate for positive inputs [14]. The output layer uses the **softmax** function, which turns the two output scores z_1 and z_2 into probabilities that are positive and sum to one:

$$softmax(z_i) = e^(z_i) / (e^(z_1) + e^(z_2))$$

The starter notebook provided with the project used a sigmoid output layer with two neurons. A sigmoid treats each output separately, so the two "probabilities" do not have to add up to one. Softmax is the correct choice with categorical cross-entropy, so the output layer was changed to softmax.

3.6.3 Loss function

Training minimises the sparse categorical cross-entropy, the negative logarithm of the probability the model assigns to the correct class, averaged over the N samples in a batch:

$$L = -(1/N) \Sigma \log p(y_i \mid x_i)$$

The loss is near zero when the model gives the correct class a probability close to one, and grows rapidly when the model is confidently wrong.

3.6.4 Optimiser: Adam

The weights were updated with Adam [16], which keeps running averages of each weight's gradient (first moment) and squared gradient (second moment) and uses them to give every weight its own adaptive step size. A learning rate of 0.001 and mini-batches of 32 samples were used.

3.6.5 Regularisation: dropout and early stopping

With only 364 training patients, over-fitting (memorising the training set instead of learning general patterns) is the main risk. Two techniques were used to reduce it. **Dropout** [15] randomly sets 20% of the first hidden layer's outputs to zero at every training step, forcing the network not to rely on any single neuron. **Early stopping** [17] monitors the validation loss after every epoch; if it has not improved for 10 consecutive epochs, training stops and the weights from the best epoch are restored.

3.7 Evaluation Metrics

Because missing a cancer is the most serious error, **malignant is treated as the positive class**. With TP, FN, FP and TN the counts of true positives, false negatives, false positives and true negatives, the metrics in Table 3.6 were computed.

Table 3.6: Evaluation metrics (malignant = positive class)

Metric	Formula	Question answered
Accuracy	(TP + TN) / (TP + TN + FP + FN)	What share of all patients is classified correctly?
Precision	TP / (TP + FP)	Of the tumours called malignant, how many really are?
Recall (sensitivity)	TP / (TP + FN)	Of all malignant tumours, how many are detected?
Specificity	TN / (TN + FP)	Of all benign tumours, how many are correctly cleared?
F1-score	2 · Precision · Recall / (Precision + Recall)	Balance between precision and recall
ROC-AUC	Area under the ROC curve	How well are malignant cases ranked above benign ones, at any threshold?

The **confusion matrix** shows the four counts directly. The **ROC curve** plots sensitivity against the false-positive rate as the decision threshold moves from 0 to 1; an area under the curve (AUC) of 1.0 means perfect separation and 0.5 means random guessing [20].

3.8 Cross-Validation and Baseline Models

A test set of 114 patients is small: a single misclassified patient changes the accuracy by 0.88 percentage points. To obtain a more reliable estimate, **stratified 5-fold cross-validation** was performed on all 569 patients: the data are split into five equal parts, and the whole training procedure (including a fresh validation split, scaling and early stopping) is

repeated five times, each time testing on a different part. The mean and standard deviation of the five accuracies are reported.

To check whether a neural network was really a good choice for this problem, four classical algorithms were also trained on exactly the same data: logistic regression, a support vector machine with RBF kernel, k-nearest neighbours ($k = 5$) and a random forest with 200 trees.

CHAPTER 4
SYSTEM DESIGN

4.1 System Architecture

Figure 4.1 shows the overall architecture. Raw data flow through cleaning, splitting and standardisation into the neural network. After evaluation, the trained model and the fitted scaler are saved to disk. The prediction system loads both, so that a new patient's measurements are transformed exactly as the training data were before being passed to the network.

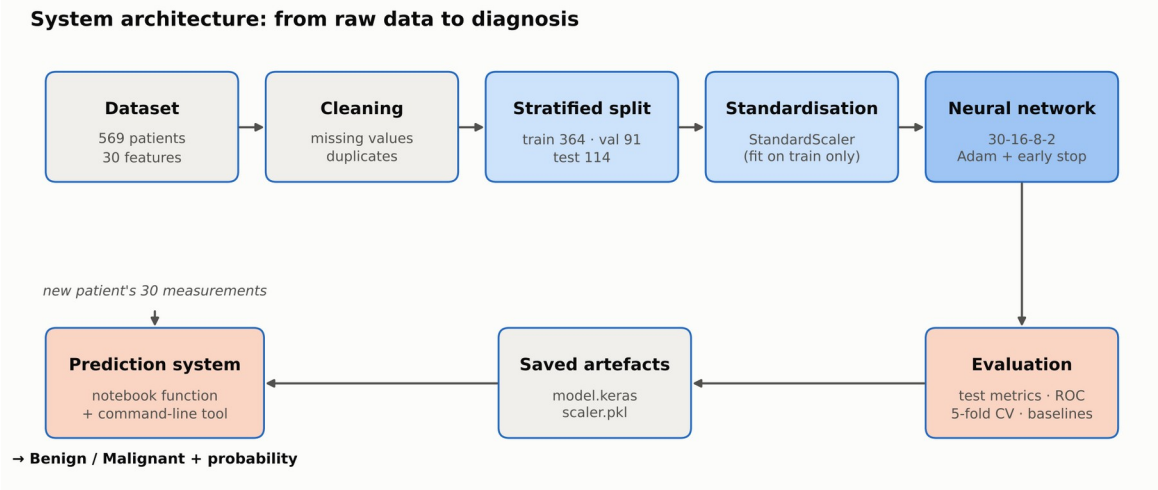


Figure 4.1: System architecture from raw data to diagnosis

4.2 Neural Network Architecture

The network is a feed-forward multilayer perceptron with two hidden layers (Figure 4.2). Table 4.1 lists each layer and its number of trainable parameters. With only 650 parameters, the model trains in a few seconds even on a normal laptop and is less likely to over-fit.

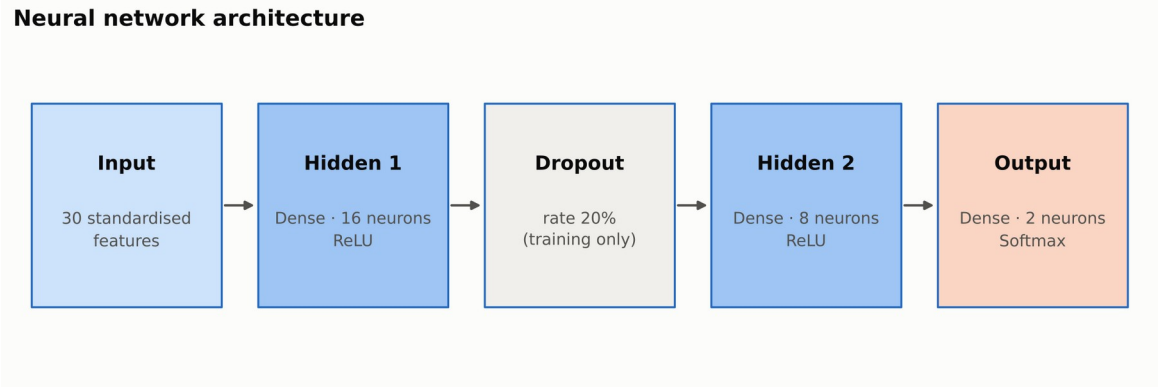


Figure 4.2: Neural network architecture

Table 4.1: Layer-by-layer structure and parameter count

Layer	Type	Output shape	Activation	Parameters
Input	Input	(None, 30)	–	0
hidden_1	Dense	(None, 16)	ReLU	30 × 16 + 16 = 496
dropout	Dropout (rate 0.2)	(None, 16)	–	0
hidden_2	Dense	(None, 8)	ReLU	16 × 8 + 8 = 136
output	Dense	(None, 2)	Softmax	8 × 2 + 2 = 18
Total				650

4.3 Hyper-parameters

Table 4.2: Training hyper-parameters

Hyper-parameter	Value	Reason
Optimiser	Adam	Works well by default for small networks
Learning rate	0.001	Keras default for Adam; stable convergence
Loss	Sparse categorical cross-entropy	Integer labels with a softmax output
Batch size	32	Standard mini-batch size
Maximum epochs	100	Upper limit; early stopping ends training sooner
Early stopping	monitor val_loss, patience 10, restore best weights	Prevents over-fitting automatically
Dropout rate	0.2 after hidden layer 1	Regularisation
Random seed	42	Reproducible results

4.4 Choice of Architecture

The starter notebook used a single hidden layer of 20 neurons. During development a few other sizes were also tried, such as two hidden layers of 32 and 16 neurons, and different dropout rates. All of them gave very similar accuracy (within about one percentage point), which is less than the normal variation between cross-validation folds. Since a bigger network did not help, a small two-layer network was chosen (16 and 8 neurons) with 20% dropout. It follows the project guide's instruction to use hidden layers, it trains quickly, and with fewer parameters it is less likely to over-fit a dataset of only 569 patients.

4.5 Software and Hardware Requirements

Table 4.3: Software requirements

Component	Version / tool	Purpose
Programming language	Python 3.10 or later	Implementation
Deep learning	TensorFlow 2.16+ with Keras	Build and train the neural network
Machine learning	scikit-learn 1.3+	Dataset, splitting, scaling, metrics, baselines

Component	Version / tool	Purpose
Data handling	NumPy, Pandas	Arrays and tables
Visualisation	Matplotlib, Seaborn	Charts
Persistence	joblib	Save the fitted scaler
Environment	Google Colab / Jupyter Notebook / VS Code	Development and execution
Version control	Git and GitHub	Code hosting and submission

Table 4.4: Hardware requirements

Component	Minimum	Used / recommended
Processor	Any 64-bit dual-core CPU	Google Colab CPU runtime
Memory	4 GB RAM	8 GB RAM or more
GPU	Not required	Not required (training takes seconds)
Storage	1 GB free (libraries)	–

4.6 Software Design and Project Structure

The code is organised as a small Python package, with one file for each part of the work (Table 4.5). The Jupyter notebook has the same steps with explanations, and it is the easiest way to run the project in Google Colab.

Table 4.5: Modules and responsibilities

File	Responsibility
src/config.py	Paths, random seed, split sizes and all hyper-parameters in one place
src/data.py	Load the dataset, quality report, cleaning, stratified split, standardisation
src/model.py	Define and compile the neural network
src/evaluate.py	Clinical metrics, classification report, ROC points, baseline models
src/visualize.py	All charts used in the notebook, report and presentation
src/train.py	End-to-end pipeline: train, evaluate, cross-validate, benchmark and save
src/predict.py	Prediction system: load model and scaler, classify new patients
notebooks/…ipynb	Step-by-step notebook for Google Colab

```
breast-cancer-classification-nn/
├── notebooks/Breast_Cancer_Classification_NN.ipynb
├── src/  config.py · data.py · model.py · evaluate.py · visualize.py · train.py · predict.py
├── models/            breast_cancer_nn.keras, scaler.pkl (created by training)
├── results/figures/   all charts;  metrics.json;  classification_report.txt
├── screenshots/       dataset, training, accuracy and prediction outputs
├── docs/              project report, synopsis, presentation
├── requirements.txt · README.md · LICENSE
```

CHAPTER 5
IMPLEMENTATION

5.1 Environment Setup

The project is written in Python using TensorFlow [21], its Keras API [22] and scikit-learn [7]. It was run in Google Colab, where all of these libraries are already installed. To run it on a local computer, the libraries can be installed from requirements.txt. The random seeds for Python, NumPy and TensorFlow are fixed and deterministic operations are turned on, so running the notebook again gives the same results.

```
pip install -r requirements.txt

import tensorflow as tf
from tensorflow import keras
SEED = 42
keras.utils.set_random_seed(SEED)               # Python, NumPy and TensorFlow
tf.config.experimental.enable_op_determinism()
```

5.2 Data Collection and Cleaning

```
from sklearn.datasets import load_breast_cancer
import pandas as pd

dataset = load_breast_cancer()
df = pd.DataFrame(dataset.data, columns=dataset.feature_names)
df["label"] = dataset.target                     # 0 = malignant, 1 = benign

print(df.isnull().sum().sum(), df.duplicated().sum())   # 0 0
df = df.dropna().drop_duplicates().reset_index(drop=True)
```

```
[2]
✓ 0s
dataset = load_breast_cancer()
df = pd.DataFrame(dataset.data, columns=dataset.feature_names)
df["label"] = dataset.target          # 0 = malignant, 1 = benign
CLASS_NAMES = ["Malignant", "Benign"]

print("Shape (rows, columns):", df.shape)
df.head()

Shape (rows, columns): (569, 31)
```

	mean radius	mean texture	mean perimeter	mean area	mean smoothness	mean compactness	mean concavity	mean concave points	mean symmetry	mean fractal dimension	...	worst texture	worst perimeter	worst area	worst smoothness	wo compactn
0	17.99	10.38	122.80	1001.0	0.11840	0.27760	0.3001	0.14710	0.2419	0.07871	...	17.33	184.60	2019.0	0.1622	0.6
1	20.57	17.77	132.90	1326.0	0.08474	0.07864	0.0869	0.07017	0.1812	0.05667	...	23.41	158.80	1956.0	0.1238	0.1
2	19.69	21.25	130.00	1203.0	0.10960	0.15990	0.1974	0.12790	0.2069	0.05999	...	25.53	152.50	1709.0	0.1444	0.4
3	11.42	20.38	77.58	386.1	0.14250	0.28390	0.2414	0.10520	0.2597	0.09744	...	26.50	98.87	567.7	0.2098	0.8
4	20.29	14.34	135.10	1297.0	0.10030	0.13280	0.1980	0.10430	0.1809	0.05883	...	16.67	152.20	1575.0	0.1374	0.2

5 rows × 31 columns

Figure 5.1: Output: first rows of the dataset (selected columns)

5.3 Data Preprocessing

```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

X, y = df.drop(columns="label"), df["label"]
X_trval, X_test, y_trval, y_test = train_test_split(
    X, y, test_size=0.20, stratify=y, random_state=SEED)
```

```
X_train, X_val, y_train, y_val = train_test_split(
    X_trval, y_trval, test_size=0.20, stratify=y_trval, random_state=SEED)

scaler = StandardScaler()
X_train_std = scaler.fit_transform(X_train)     # learn mean/std on training data
only
X_val_std, X_test_std = scaler.transform(X_val), scaler.transform(X_test)
```

5.4 Building the Neural Network

```
def build_model(n_features=30):
    model = keras.Sequential([
        keras.layers.Input(shape=(n_features,)),
        keras.layers.Dense(16, activation="relu", name="hidden_1"),
        keras.layers.Dropout(0.2, name="dropout"),
        keras.layers.Dense(8, activation="relu", name="hidden_2"),
        keras.layers.Dense(2, activation="softmax", name="output"),
    ], name="breast_cancer_nn")
    model.compile(optimizer=keras.optimizers.Adam(learning_rate=0.001),
                  loss="sparse_categorical_crossentropy", metrics=["accuracy"])
    return model
```

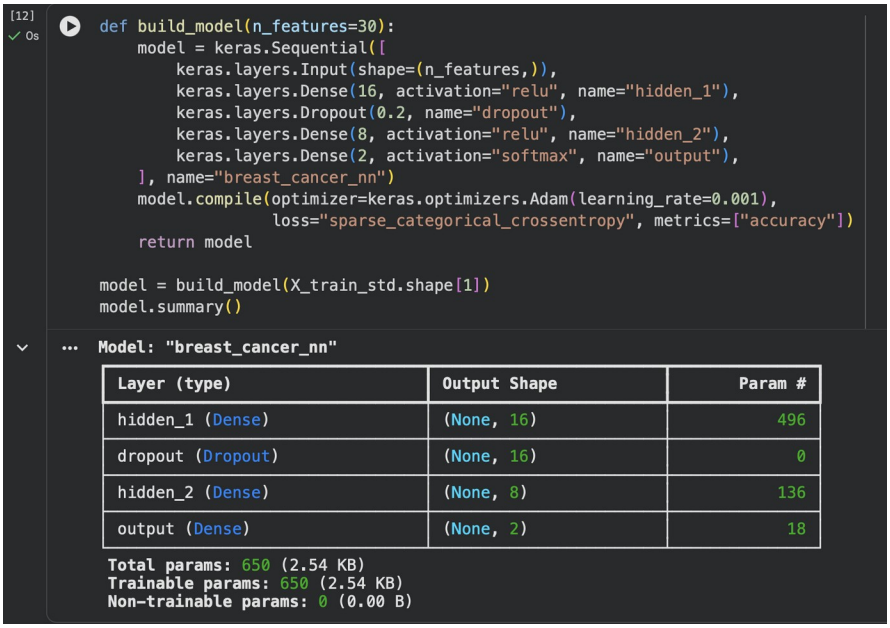


Figure 5.2: Output: model summary

5.5 Training the Model

```
early_stop = keras.callbacks.EarlyStopping(monitor="val_loss", patience=10,
                                           restore_best_weights=True, verbose=1)
history = model.fit(X_train_std, y_train, validation_data=(X_val_std, y_val),
                    epochs=100, batch_size=32, callbacks=[early_stop], verbose=2)
```

```
[13]
✓ 9s
early_stop = keras.callbacks.EarlyStopping(monitor="val_loss", patience=10,
                                           restore_best_weights=True, verbose=1)

history = model.fit(X_train_std, y_train,
                    validation_data=(X_val_std, y_val),
                    epochs=100, batch_size=32,
                    callbacks=[early_stop], verbose=2)

...  Epoch 1/100
12/12 - 2s - 147ms/step - accuracy: 0.5275 - loss: 0.6988 - val_accuracy: 0.5495 - val_loss: 0.6045
Epoch 2/100
12/12 - 0s - 10ms/step - accuracy: 0.6346 - loss: 0.5787 - val_accuracy: 0.6923 - val_loss: 0.5104
Epoch 3/100
12/12 - 0s - 10ms/step - accuracy: 0.7115 - loss: 0.4885 - val_accuracy: 0.8022 - val_loss: 0.4383
Epoch 4/100
12/12 - 0s - 10ms/step - accuracy: 0.7775 - loss: 0.4234 - val_accuracy: 0.8901 - val_loss: 0.3805
Epoch 5/100
12/12 - 0s - 10ms/step - accuracy: 0.8626 - loss: 0.3706 - val_accuracy: 0.9451 - val_loss: 0.3343

...

Epoch 44/100
12/12 - 0s - 15ms/step - accuracy: 0.9918 - loss: 0.0485 - val_accuracy: 0.9670 - val_loss: 0.0818
Epoch 45/100
12/12 - 0s - 15ms/step - accuracy: 0.9918 - loss: 0.0504 - val_accuracy: 0.9670 - val_loss: 0.0827
Epoch 46/100
12/12 - 0s - 17ms/step - accuracy: 0.9863 - loss: 0.0409 - val_accuracy: 0.9670 - val_loss: 0.0838
Epoch 47/100
12/12 - 0s - 17ms/step - accuracy: 0.9890 - loss: 0.0401 - val_accuracy: 0.9670 - val_loss: 0.0844
Epoch 48/100
12/12 - 0s - 24ms/step - accuracy: 0.9890 - loss: 0.0538 - val_accuracy: 0.9670 - val_loss: 0.0853
Epoch 49/100
12/12 - 0s - 27ms/step - accuracy: 0.9835 - loss: 0.0504 - val_accuracy: 0.9670 - val_loss: 0.0854
Epoch 50/100
12/12 - 0s - 14ms/step - accuracy: 0.9945 - loss: 0.0440 - val_accuracy: 0.9670 - val_loss: 0.0849
Epoch 51/100
12/12 - 0s - 17ms/step - accuracy: 0.9835 - loss: 0.0500 - val_accuracy: 0.9670 - val_loss: 0.0828
Epoch 51: early stopping
Restoring model weights from the end of the best epoch: 41.
```

Figure 5.3: Output: training log (first and last epochs)

5.6 Evaluation

The model outputs two probabilities for every patient. np.argmax selects the class with the higher probability, and scikit-learn computes the metrics with malignant (label 0) as the positive class.

```
probs = model.predict(X_test_std)               # [[P(malignant), P(benign)], ...]
y_pred = np.argmax(probs, axis=1)
p_malignant = probs[:, 0]

precision = precision_score(y_test, y_pred, pos_label=0)
recall    = recall_score(y_test, y_pred, pos_label=0)
f1        = f1_score(y_test, y_pred, pos_label=0)
auc       = roc_auc_score((y_test == 0).astype(int), p_malignant)
cm        = confusion_matrix(y_test, y_pred, labels=[0, 1])
```

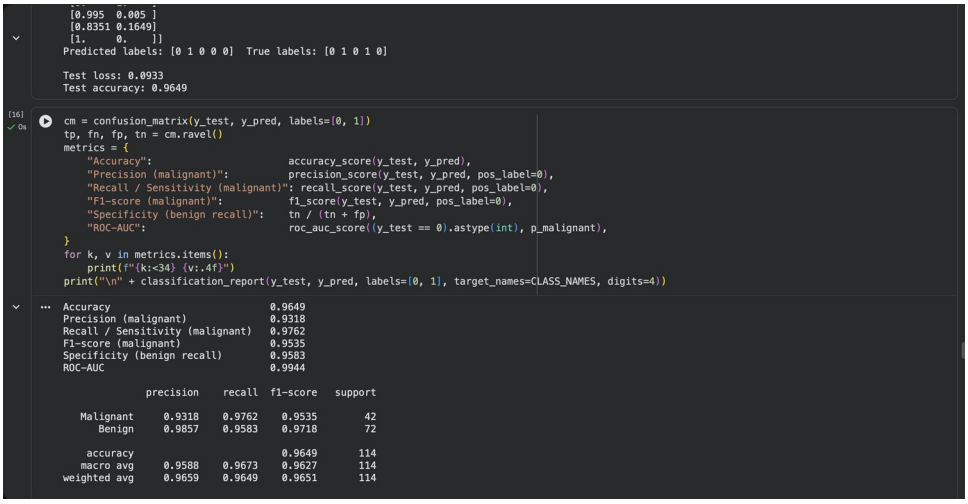


Figure 5.4: Output: test metrics and classification report

5.7 Predictive System

The prediction function applies the saved scaler to the 30 raw measurements of a new patient, runs the network and reports the diagnosis with its probability. The same logic is available from the command line in src/predict.py, which also accepts a CSV file containing many patients.

```
def predict_tumour(measurements):
    x = pd.DataFrame([measurements], columns=X.columns)
    p = model.predict(scaler.transform(x), verbose=0)[0]
    label = int(np.argmax(p))
    print(f"Prediction: {CLASS_NAMES[label].upper()} (confidence {p[label]:.1%})")
    return CLASS_NAMES[label]

# Command line
#   python -m src.train
#   python -m src.predict --values "17.99,10.38,122.8, ... ,0.1189"
#   python -m src.predict --csv new_patients.csv
```



Figure 5.5: Output: predictions for a benign and a malignant sample

5.8 Saving the Model

The trained network is saved in the native Keras format (breast_cancer_nn.keras) and the fitted scaler with joblib (scaler.pkl). Both files are needed for prediction, because the model

only gives correct answers when new inputs are scaled in exactly the same way as the training data.

CHAPTER 6
RESULTS AND DISCUSSION

6.1 Training Behaviour

Figure 6.1 shows accuracy and loss on the training and validation sets after each epoch. Both losses fall steeply during the first ten epochs and then level off. The validation loss reached its minimum at epoch 41; because it did not improve for the next 10 epochs, early stopping ended training at epoch 51 and restored the weights from epoch 41. At that point training accuracy was 98.6% and validation accuracy 96.7%, with losses of 0.057 and 0.080. The training and validation curves stay close together, which shows that the model is learning general patterns and is not over-fitting.

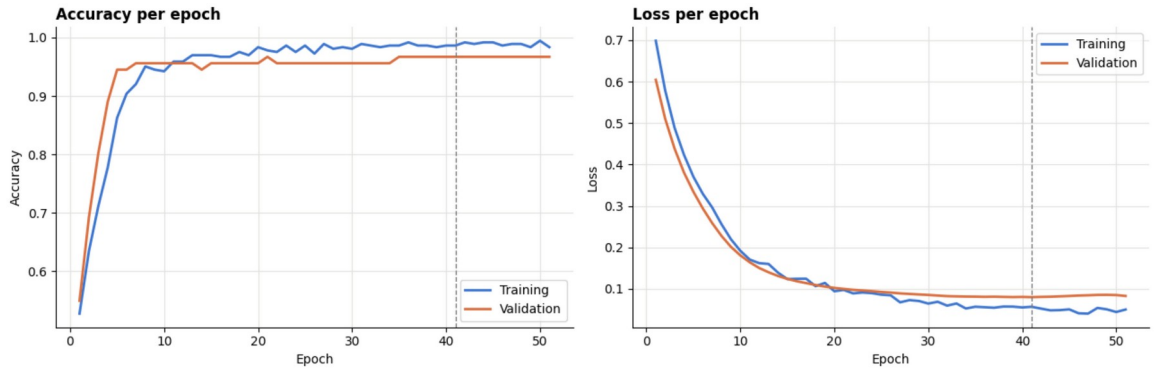


Figure 6.1: Training and validation accuracy and loss per epoch

6.2 Performance on the Test Set

On the 114 test patients, which were never used for training or design decisions, the network achieved an accuracy of 96.49%, well above the 90% target set in the project guide. Table 6.1 summarises all metrics.

Table 6.1: Test-set performance of the neural network

Metric	Value
Accuracy	96.49%
Precision (malignant)	93.18%
Recall / sensitivity (malignant)	97.62%
F1-score (malignant)	95.35%
Specificity (benign recall)	95.83%
ROC-AUC	0.9944
Test loss	0.0933

Table 6.2: Classification report per class

Class	Precision	Recall	F1-score	Support
Malignant	93.18%	97.62%	95.35%	42
Benign	98.57%	95.83%	97.18%	72
Macro average	95.88%	96.73%	96.27%	114

6.3 Confusion Matrix

The confusion matrix (Figure 6.2) shows that the model correctly identified 41 of the 42 malignant tumours and 69 of the 72 benign tumours. Only 1 malignant tumour was missed (false negative), while 3 benign tumours were flagged as malignant (false positives). In a screening context this balance is desirable: a false positive leads to further tests, which is inconvenient but safe, whereas a false negative could delay treatment.

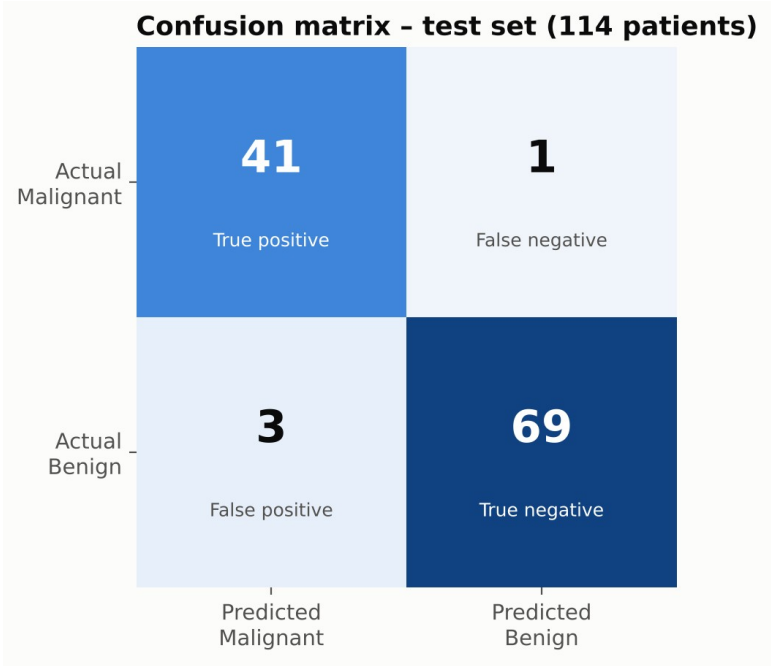


Figure 6.2: Confusion matrix on the test set

6.4 ROC Curve

The ROC curve (Figure 6.3) rises almost vertically along the left edge and reaches the top of the plot at a very low false-positive rate. The area under the curve is 0.994, meaning that for a randomly chosen malignant and benign pair the model assigns the higher malignancy probability to the malignant tumour about 99.4% of the time. Because the AUC is so high, the decision threshold could be lowered to catch even more cancers, at the cost of a few more false alarms.

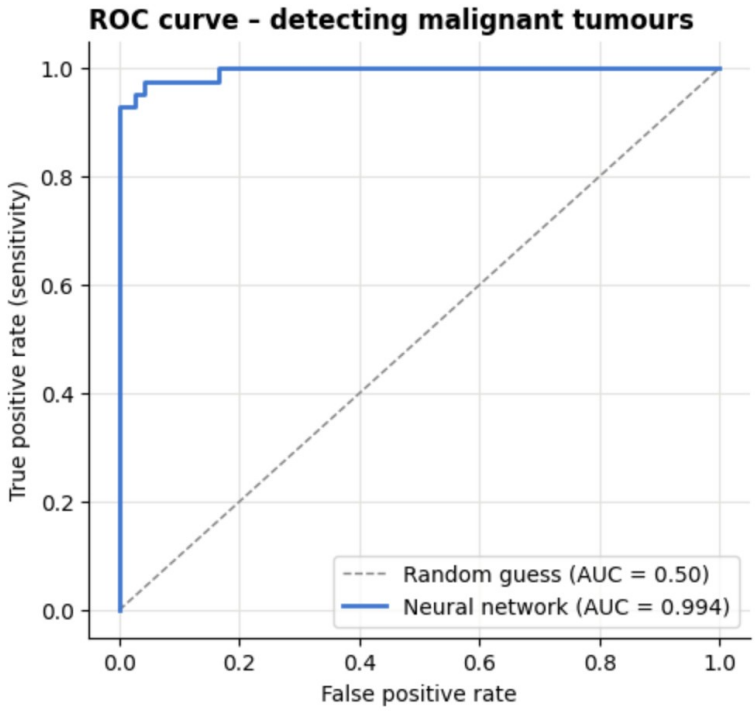


Figure 6.3: ROC curve for detecting malignant tumours

6.5 Prediction Confidence and Error Analysis

Figure 6.4 shows the predicted probability of malignancy for every test patient. Most predictions are very confident: 104 of the 114 patients (91%) received a probability below 0.1 or above 0.9, and 103 of those 104 predictions were correct. Table 6.3 lists the four misclassified patients. Three of them, all benign tumours flagged as malignant, had probabilities between 0.57 and 0.84, so the model was relatively uncertain about them. The fourth error is more serious: one malignant tumour (test index 53) received a malignancy probability of only 0.034 and was confidently classified as benign. A confident prediction is therefore not a guarantee, and the model should be used as a second reader that supports, rather than replaces, a specialist.

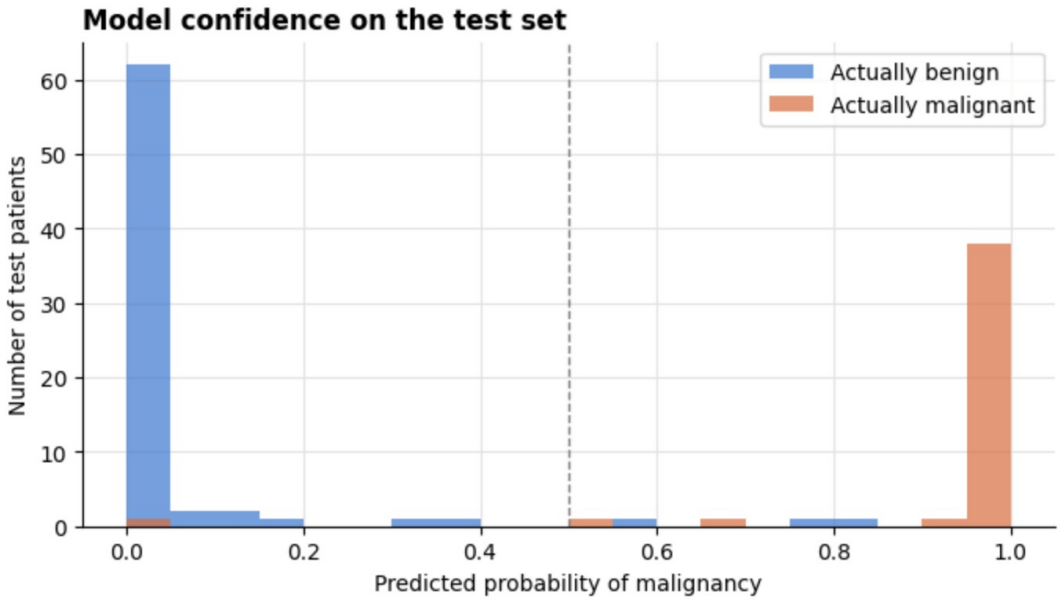


Figure 6.4: Distribution of predicted malignancy probabilities on the test set

Table 6.3: Misclassified test patients

Test index	True class	Predicted class	P(malignant)
3	Benign	Malignant	0.835
16	Benign	Malignant	0.782
53	Malignant	Benign	0.034
66	Benign	Malignant	0.566

6.6 Comparison with Classical Machine Learning

Table 6.4 compares the neural network with the four baseline models, all trained on the same 455 patients and evaluated on the same 114 test patients.

Table 6.4: Test-set comparison with baseline models

Model	Accuracy	Precision (M)	Recall (M)	F1 (M)	ROC-AUC
Neural Network (ours)	**96.49%**	**93.18%**	**97.62%**	**95.35%**	**0.994**
Logistic Regression	98.25%	97.62%	97.62%	97.62%	0.996
Support Vector Machine (RBF)	98.25%	97.62%	97.62%	97.62%	0.995
K-Nearest Neighbours (k=5)	95.61%	95.12%	92.86%	93.98%	0.979
Random Forest (200 trees)	94.74%	92.86%	92.86%	92.86%	0.994

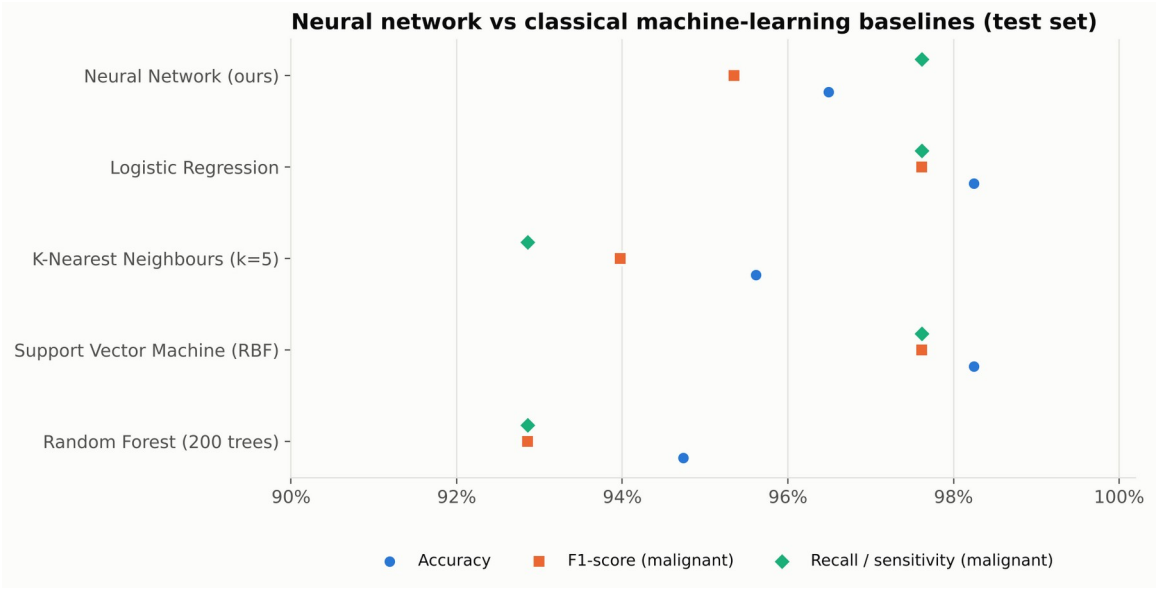


Figure 6.5: Accuracy, F1-score and sensitivity of all models on the test set

The neural network ties with logistic regression and the SVM for the highest sensitivity (97.62%): all three detect 41 of the 42 cancers. Logistic regression and the SVM made two fewer false alarms, so their test accuracy is slightly higher (98.25%). K-nearest neighbours and the random forest each missed three cancers.

6.7 Cross-Validation Results

Because a difference of two patients on a 114-patient test set is within the range of chance, 5-fold cross-validation over all 569 patients was used to compare the models more reliably (Table 6.5).

Table 6.5: 5-fold stratified cross-validation accuracy (all 569 patients)

Model	Fold 1	Fold 2	Fold 3	Fold 4	Fold 5	Mean ± std
Neural Network (ours)	**98.3%**	**96.5%**	**97.4%**	**97.4%**	**99.1%**	**97.72% ± 0.89%**
Logistic Regression	97.4%	94.7%	96.5%	99.1%	99.1%	97.37% ± 1.66%
Support Vector Machine (RBF)	99.1%	94.7%	97.4%	99.1%	98.2%	97.72% ± 1.63%
K-Nearest Neighbours (k=5)	98.2%	94.7%	93.9%	98.2%	96.5%	96.31% ± 1.79%
Random Forest (200 trees)	96.5%	93.9%	95.6%	94.7%	96.5%	95.43% ± 1.02%

Under cross-validation the neural network reaches a mean accuracy of 97.72%, equal to the SVM (97.72%) for the highest of all five models and ahead of logistic regression (97.37%). Its standard deviation (0.89%) is the lowest of all five models, meaning that its performance is the most consistent across different subsets of patients. However, the differences between the top three models are smaller than their standard deviations, so none of them can be called clearly better.

6.8 Comparison with Published Work

Table 6.6: Comparison with a published study on the same dataset

Study	Evaluation protocol	Model	Accuracy
Agarap, 2018 [8]	Single 70/30 split	Multilayer perceptron	99.04%
Agarap, 2018 [8]	Single 70/30 split	Softmax regression	97.66%
Agarap, 2018 [8]	Single 70/30 split	SVM	96.09%
This project	Single 80/20 split (test)	Neural network 30-16-8-2	96.49%
This project	5-fold cross-validation	Neural network 30-16-8-2	97.72%

The results are in the same range as those reported in the literature. Exact numbers are not directly comparable because each study uses a different split of the data and a different evaluation protocol; as the cross-validation folds in Table 6.5 show, the accuracy of the same model varies between about 96.5% and 99.1% depending on which patients are in the test set.

6.9 Discussion

- **The target was met easily.** Test accuracy of 96.49% and cross-validated accuracy of 97.72% both exceed the 90% requirement.
- **Sensitivity is high.** 97.62% of malignant tumours were detected, which is the most important property for a diagnostic aid.
- **Size and shape are the key signals.** Radius, perimeter, area, concavity and concave points separate the two classes most clearly. This matches what cytopathologists look for under the microscope.
- **Most errors are borderline, but not all.** Three of the four misclassified patients had probabilities between 0.57 and 0.84, close to the threshold; the one missed cancer received a probability of only 0.034, which shows why the model should support, not replace, expert review.
- **Simple models are competitive.** On a small, well-separated dataset like this one, logistic regression and the SVM do just as well as the neural network. Deep learning is more useful with bigger datasets and raw images, as in [4] and [5]. This was one of the more interesting results of the comparison.

6.10 Limitations

- The dataset contains only 569 patients from a single institution, collected in the early 1990s; performance may differ for other populations, laboratories or imaging equipment.
- The model uses pre-computed nucleus measurements and therefore depends on the image-analysis step that produced them.

- The test set is small, so test-set metrics carry an uncertainty of roughly ±2 percentage points.
- The model has not been clinically validated and must not be used for real diagnosis.

CHAPTER 7

CONCLUSION

This project developed a neural-network system that classifies breast tumours as benign or malignant from 30 measurements of cell nuclei. Following the seven phases in the Naviotech project guide, the Breast Cancer Wisconsin (Diagnostic) dataset was explored, cleaned, split into stratified training, validation and test sets, and standardised without data leakage. A compact feed-forward network with two hidden layers (16 and 8 ReLU neurons), dropout and a softmax output (650 parameters in total) was trained with the Adam optimiser and early stopping.

On 114 unseen test patients the model achieved an accuracy of 96.49%, a sensitivity of 97.62% (41 of 42 cancers detected), a precision of 93.18%, an F1-score of 95.35% and a ROC-AUC of 0.994. Five-fold cross-validation confirmed a mean accuracy of 97.72% with the lowest variability of all models tested. So the project's target of more than 90% accuracy was clearly met.

The comparison with four classical algorithms showed that logistic regression and the SVM work just as well on this small dataset, while the neural network gave the most consistent results across the cross-validation folds. The exploratory analysis confirmed that larger and more irregular nuclei are the strongest indicators of malignancy.

Apart from the results, this project gave me hands-on practice with the whole machine-learning process: exploring data, preprocessing without leakage, designing and training a neural network, using dropout and early stopping, choosing the right evaluation metrics for a medical problem, comparing with simpler models, organising code properly and using GitHub. One thing I learned is that accuracy alone is not enough. For a medical problem, it matters a lot which mistakes the model makes, and that is why I looked closely at the missed cancer.

CHAPTER 8

FUTURE SCOPE

- **Larger and more diverse data.** Training and validating on multi-centre datasets from different populations would show whether the model generalises beyond a single hospital.
- **Explainability.** Methods such as SHAP values could show which measurements led to each prediction, so that doctors can understand why the model made a decision.
- **Threshold tuning.** The high ROC-AUC means that lowering the decision threshold could raise sensitivity further, with a controlled increase in false positives.
- **Flagging uncertain cases.** Predictions with probabilities near 0.5 could be sent automatically for expert review. This would not have caught the one confident miss in this project, so it should be combined with other checks.
- **Image-based deep learning.** Convolutional neural networks trained directly on digitised histopathology or cytology images could remove the need for hand-crafted measurements.
- **Hyper-parameter optimisation.** Automated search tools such as KerasTuner could explore learning rates, layer sizes and regularisation more systematically.
- **Deployment.** A web application built with Streamlit or Flask could let users enter measurements and receive a prediction with an explanation, as a teaching or decision-support tool.

REFERENCES

[1] World Health Organization, "Global breast cancer survival estimates advance WHO Global Breast Cancer Initiative," Knowledge Action Portal on NCDs, 8 Jul. 2026. [Online]. Available: https://knowledge-action-portal.com/en/content/global-breast-cancer-survival-estimates-advance-who-global-breast-cancer-initiative

[2] W. N. Street, W. H. Wolberg and O. L. Mangasarian, "Nuclear feature extraction for breast tumor diagnosis," in Proc. IS&T/SPIE Int. Symp. Electronic Imaging: Science and Technology, vol. 1905, pp. 861–870, 1993.

[3] W. Wolberg, O. Mangasarian, N. Street and W. Street, "Breast Cancer Wisconsin (Diagnostic)," UCI Machine Learning Repository, 1995. doi: 10.24432/C5DW2B.

[4] A. Esteva, B. Kuprel, R. A. Novoa, J. Ko, S. M. Swetter, H. M. Blau and S. Thrun, "Dermatologist-level classification of skin cancer with deep neural networks," Nature, vol. 542, pp. 115–118, 2017.

[5] S. M. McKinney et al., "International evaluation of an AI system for breast cancer screening," Nature, vol. 577, pp. 89–94, 2020.

[6] K. P. Bennett and O. L. Mangasarian, "Robust linear programming discrimination of two linearly inseparable sets," Optimization Methods and Software, vol. 1, pp. 23–34, 1992.

[7] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," Journal of Machine Learning Research, vol. 12, pp. 2825–2830, 2011.

[8] A. F. M. Agarap, "On breast cancer detection: An application of machine learning algorithms on the Wisconsin diagnostic dataset," in Proc. 2nd Int. Conf. Machine Learning and Soft Computing (ICMLSC), Phu Quoc, Vietnam, 2018. doi: 10.1145/3184066.3184080.

[9] C. Cortes and V. Vapnik, "Support-vector networks," Machine Learning, vol. 20, no. 3, pp. 273–297, 1995.

[10] T. Cover and P. Hart, "Nearest neighbor pattern classification," IEEE Transactions on Information Theory, vol. 13, no. 1, pp. 21–27, 1967.

[11] L. Breiman, "Random forests," Machine Learning, vol. 45, no. 1, pp. 5–32, 2001.

[12] D. E. Rumelhart, G. E. Hinton and R. J. Williams, "Learning representations by back-propagating errors," Nature, vol. 323, pp. 533–536, 1986.

[13] K. Hornik, M. Stinchcombe and H. White, "Multilayer feedforward networks are universal approximators," Neural Networks, vol. 2, no. 5, pp. 359–366, 1989.

[14] V. Nair and G. E. Hinton, "Rectified linear units improve restricted Boltzmann machines," in Proc. 27th Int. Conf. Machine Learning (ICML), 2010, pp. 807–814.

[15] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever and R. Salakhutdinov, "Dropout: A simple way to prevent neural networks from overfitting," Journal of Machine Learning Research, vol. 15, pp. 1929–1958, 2014.

[16] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in Proc. 3rd Int. Conf. Learning Representations (ICLR), 2015.

[17] L. Prechelt, "Early stopping – but when?" in Neural Networks: Tricks of the Trade, G. B. Orr and K.-R. Müller, Eds. Berlin: Springer, 1998, pp. 55–69.

[18] Y. LeCun, Y. Bengio and G. Hinton, "Deep learning," Nature, vol. 521, pp. 436–444, 2015.

[19] I. Goodfellow, Y. Bengio and A. Courville, Deep Learning. Cambridge, MA: MIT Press, 2016.

[20] T. Fawcett, "An introduction to ROC analysis," Pattern Recognition Letters, vol. 27, no. 8, pp. 861–874, 2006.

[21] M. Abadi et al., "TensorFlow: A system for large-scale machine learning," in Proc. 12th USENIX Symp. Operating Systems Design and Implementation (OSDI), 2016, pp. 265–283.

[22] F. Chollet et al., "Keras," 2015. [Online]. Available: https://keras.io

APPENDIX A: SOURCE CODE

The complete project, including the Jupyter notebook, training pipeline and visualisation code, is available on GitHub at github.com/sangini-agrawal/breast-cancer-classification-nn. The main modules are reproduced below.

A.1 src/model.py

```
"""Neural network architecture (Phase 4)."""
from tensorflow import keras

from . import config


def build_model(n_features: int) -> keras.Sequential:
    """Feed-forward neural network: 30 inputs -> 32 ReLU -> Dropout -> 16 ReLU ->
2 Softmax.

    * ReLU in the hidden layers lets the network learn non-linear patterns.
    * Dropout randomly switches off 20 % of neurons during training to reduce
over-fitting.
    * Softmax in the output layer turns the two scores into probabilities that sum
to 1
      (P(malignant) + P(benign) = 1), which is the correct pairing for
      sparse categorical cross-entropy loss.
    """
    h1, h2 = config.HIDDEN_UNITS
    model = keras.Sequential(
        [
            keras.layers.Input(shape=(n_features,)),
            keras.layers.Dense(h1, activation="relu", name="hidden_1"),
            keras.layers.Dropout(config.DROPOUT_RATE, name="dropout"),
            keras.layers.Dense(h2, activation="relu", name="hidden_2"),
            keras.layers.Dense(2, activation="softmax", name="output"),
        ],
        name="breast_cancer_nn",
    )
    model.compile(
        optimizer=keras.optimizers.Adam(learning_rate=config.LEARNING_RATE),
        loss="sparse_categorical_crossentropy",
        metrics=["accuracy"],
    )
    return model
```

A.2 src/predict.py

```
"""Phase 7 – Predictive system.

Classify a tumour as benign or malignant from its 30 measurements.

Examples (run from the project root after `python -m src.train`):
    python -m src.predict                          # runs the built-in demo sample
    python -m src.predict --values "11.76,21.6,74.72, ... ,0.06563"
    python -m src.predict --csv new_patients.csv  # CSV with the 30 feature
columns
```

```
"""
from __future__ import annotations

import argparse
import os

os.environ.setdefault("TF_CPP_MIN_LOG_LEVEL", "2")

import joblib
import numpy as np
import pandas as pd
from tensorflow import keras

from . import config

# Measurements of one benign sample from the dataset (used for the demo)
DEMO_SAMPLE = (11.76, 21.6, 74.72, 427.9, 0.08637, 0.04966, 0.01657, 0.01115,
0.1495, 0.05888,
               0.4062, 1.21, 2.635, 28.47, 0.005857, 0.009758, 0.01168, 0.007445,
0.02406, 0.001769,
               12.98, 25.72, 82.98, 516.5, 0.1085, 0.08615, 0.05523, 0.03715,
0.2433, 0.06563)


def load_artifacts():
    if not config.MODEL_PATH.exists() or not config.SCALER_PATH.exists():
        raise SystemExit("Model not found. Train it first with:  python -m
src.train")
    return keras.models.load_model(config.MODEL_PATH),
joblib.load(config.SCALER_PATH)


def predict(values, model=None, scaler=None) -> list[dict]:
    """values: array-like of shape (n_samples, 30) in the dataset's original
units."""
    if model is None or scaler is None:
        model, scaler = load_artifacts()
    X = np.asarray(values, dtype=float).reshape(-1, 30)
    if hasattr(scaler, "feature_names_in_"):
        X = pd.DataFrame(X, columns=scaler.feature_names_in_)  # avoids feature-
name warning
    probs = model.predict(scaler.transform(X), verbose=0)
    out = []
    for p in probs:
        label = int(np.argmax(p))
        out.append({"diagnosis": config.CLASS_NAMES[label],
                    "p_malignant": float(p[config.MALIGNANT]), "p_benign":
float(p[config.BENIGN]),
                    "confidence": float(p[label])})
    return out


def main():
    ap = argparse.ArgumentParser(description="Breast tumour classifier (neural
network)")
    ap.add_argument("--values", help="30 comma-separated measurements for one
patient")
```

```
    ap.add_argument("--csv", help="CSV file with one patient per row and the 30
feature columns")
    args = ap.parse_args()

    if args.csv:
        rows = pd.read_csv(args.csv).to_numpy()
    elif args.values:
        rows = [[float(v) for v in args.values.split(",")]]
    else:
        print("No input given – running the demo sample.")
        rows = [DEMO_SAMPLE]

    for i, r in enumerate(predict(rows), 1):
        print(f"Patient {i}: the tumour is predicted {r['diagnosis'].upper()} "
              f"(confidence {r['confidence']:.1%}; P(malignant) =
{r['p_malignant']:.3f})")
    print("\nNote: this is an educational model and not a medical diagnosis.")


if __name__ == "__main__":
    main()
```